

**PONTIFICIA UNIVERSIDAD CATÓLICA
DEL ECUADOR SEDE MANABÍ**

PUCE – MANABÍ

Facultad de Ingeniería
Carrera de Ingeniería en Software

[INFORME PROYECTO FINAL]



DATOS DEL ESTUDIANTE	
Estudiante:	Diego Fernando Loor Rivas
Semestre:	Quinto Semestre (5to)
Docente:	Jose Luis Naranhi
Materia:	Calidad de Software
Fecha:	Manta, agosto de 2026

Repositorio del proyecto

<https://github.com/Manta360/Manta360>

Despliegue

<https://aunfalta.com>

Reporte de Gestión e Historial Técnico

Resumen de tickets gestionados en Jira

Mi participación se enfocó en la integración, sincronización y validación de la base de datos remota del proyecto Manta360 mediante Supabase y Prisma.

ID	Descripción	Estado
DIEGO	Diagnóstico del error de permisos sobre la tabla identity_documents.	Finalizado
DIEGO	Corrección de ownership y creación de la columna side.	Finalizado
DIEGO	Sincronización de tablas y columnas faltantes entre Prisma y Supabase.	Finalizado
DIEGO	Verificación de Storage, usuario Municipio y conexión sin Docker.	Finalizado

Aportes técnicos en código (commits y Pull Requests)

Autor Git: Diego Fernando Loor Rivas

Correo Git: [DILLOOR2000]

Problema detectado: permisos de identity_documents

Durante la sincronización del esquema se identificó el siguiente error de PostgreSQL

must be owner of table identity_documents

El problema ocurría porque el rol prisma tenía permisos para consultar y modificar datos, pero no era propietario de la tabla. PostgreSQL exige ser propietario de una tabla para aplicar cambios estructurales mediante ALTER TABLE.

La solución aplicada fue transferir la propiedad de la tabla al rol prisma y agregar la columna requerida por el modelo:

```
ALTER TABLE public.identity_documents OWNER TO prisma;
```

```
ALTER TABLE public.identity_documents
```

```
ADD COLUMN IF NOT EXISTS "side" varchar(12)
```

```
NOT NULL DEFAULT 'UNICA';
```

Con este cambio se corrigió el error de permisos y la tabla quedó preparada para futuras migraciones realizadas por Prisma.

Sincronización de Prisma con Supabase

También se detectó que Prisma esperaba columnas y tablas que todavía no estaban creadas en Supabase. Se añadieron los siguientes elementos:

ALTER TABLE public.users

ADD COLUMN IF NOT EXISTS "nationalId" text;

ALTER TABLE public.properties

ADD COLUMN IF NOT EXISTS approved boolean NOT NULL DEFAULT false,

ADD COLUMN IF NOT EXISTS "approvedAt" timestamp(3),

ADD COLUMN IF NOT EXISTS "approvedBy" text;

Además, se sincronizaron las tablas:

- chat_messages
- contract_renewal_requests
- identity_documents
- properties
- users

Se verificó que Supabase contiene las tablas requeridas por el modelo y los buckets de Storage:

- identity-documents: privado, porque contiene documentos sensibles.
- property-images: público, para mostrar imágenes de inmuebles.

Validación de infraestructura

Se comprobó que la aplicación funciona con Docker apagado. Esto demuestra que la base de datos utilizada no es local, sino que está alojada en Supabase remoto.

La aplicación necesita conservar sus variables de entorno para conectarse:

Propuestas y Aprendizaje

Propuesta aportada

Migraciones versionadas

Se propone que los cambios futuros de base de datos se manejen mediante migraciones versionadas y revisadas con Pull Requests, en lugar de depender únicamente de prisma db push.

Cada migración debería incluir:

- Nombre descriptivo.
- Script SQL reproducible.
- Revisión mediante Pull Request.
- Validación previa en Supabase.
- Evidencia de ejecución.
- Revisión de permisos y RLS.

Esta propuesta evita que Prisma y Supabase tengan estructuras diferentes y facilita recuperar cambios si ocurre algún problema.

Seguridad por capas

Se propone documentar claramente qué partes del proyecto usan Prisma, cuáles usan Supabase Storage y cuáles podrían utilizar el Data API.

La información sensible, como los documentos de identidad, debe mantenerse en buckets privados. También es necesario evitar exponer claves como `SUPABASE_SERVICE_ROLE_KEY`, ya que esta clave tiene privilegios elevados y solo debe utilizarse desde el servidor.

Checklist antes de publicar cambios

Antes de ejecutar cambios importantes se debe verificar:

- Variables de entorno configuradas.
- Conexión correcta con Supabase.
- Tablas y columnas requeridas por Prisma.
- Buckets de Storage existentes.
- Inicio de sesión funcionando.
- Consulta de propiedades funcionando.
- Carga de imágenes y documentos funcionando.
- Ausencia de secretos en GitHub.

Conclusión reflexiva

Este aporte me permitió comprender la relación entre una aplicación Next.js, Prisma y Supabase. Aprendí que Supabase permite usar una base de datos remota sin depender de Docker o PostgreSQL local; sin embargo, la aplicación todavía necesita variables de entorno para conocer la URL y las credenciales de conexión.

También aprendí a diferenciar tres conceptos importantes:

- **Ownership:** permite modificar la estructura de una tabla.

- **Permisos SQL:** permiten consultar, insertar, actualizar o eliminar datos.
- **RLS:** controla qué filas puede consultar o modificar cada usuario.

Finalmente, comprendí que una integración correcta no consiste únicamente en conectar la aplicación a Supabase, sino en mantener sincronizado el esquema de Prisma, proteger las credenciales, verificar Storage y comprobar que los flujos principales funcionen correctamente.